LLM MODEL-RISK MONITORING PLATFORM

Phase 9–12 Roadmap: Real-Signal Integration, Verification, Production Deployment & Portfolio Packaging

Prepared from a direct audit of the repository — every finding below was read and, where possible, executed in the code, not inferred from the project summary.

Domain: Finance & Banking | Governance Frame: Federal Reserve SR 11-7

Contents

(The table above is a live Word field — right-click it and choose "Update Field" to populate page numbers if it renders empty. It only auto-builds inside Microsoft Word; if you're viewing this in another app, use the manual list below instead.)

How to Read This Document

This roadmap picks up immediately after the eight phases described in the existing project summary. Before proposing what comes next, the current repository was audited directly — every module was opened and read, the test suite was executed, coverage was measured, mypy was run, and the CI workflow files were traced line by line against the code they reference. The findings below are what that audit actually showed, not a restatement of the summary's claims.

This matters because the summary and the codebase tell two different stories in one specific place. The summary describes a finished evaluation and monitoring system. The code underneath it is a genuinely well-engineered piece of infrastructure — the database schema, the statistics, the tracing pipeline, and the CI scaffolding are real, correct, and tested — built around three components that are explicitly labeled STUB in the source and return fabricated data. The architecture is not the gap. The signal running through it is.

> **WHY THIS CHANGES THE PLAN**
>
> A generic "next phase" plan for a project like this would jump straight to load testing, Kubernetes manifests, and a demo video. That plan would be wrong here, because right now the regression gate this project is supposed to deliver cannot fail — not due to a bug, but because the CI workflow compares one file to itself, and the score that would detect a difference is generated by a function that ignores its input. Phase 9 exists to fix that specific problem before anything else. Phases 10–12 assume Phase 9 is done.

Structure

- Section 1 — **Verified Current-State Assessment**: what was actually found, module by module, with severity ratings.
- Section 2 — **Phase 9: Real-Signal Integration** (the requested "next-to-final" development phase): closes the stub chain, wires the dashboards to the real database, and fixes the infrastructure gaps found during the audit.
- Section 3 — **Phase 10: Verification & Testing**: integration tests, load and chaos testing, and a security pass — the testing layer requested.
- Section 4 — **Phase 11: Production Deployment**: takes the verified system from a laptop to a real, reachable environment.
- Section 5 — **Phase 12: Resume & Portfolio Packaging**: README, live demo, and — critically — an honest, defensible way to describe this project in an interview.
- Appendix — **consolidated checklist and file-by-file change list** for quick reference while implementing.

1. Verified Current-State Assessment

Phases 1 through 8, as described in the project summary, are reflected in the repository. What follows is a precise accounting of what each phase actually contains today, verified by reading the code and, wherever feasible, running it. Findings are grouped by severity, not by phase, because the most important issue in the repository cuts across three phases at once.

1.1 Scorecard

Area	Claimed (Summary)	Verified Status	Evidence
Evidence DB & schema	Append-only, audit-sound	**Solid**	12 normalized tables, correct FKs; no updated_at on evidence tables
Async tracing / @traced_call	Captures full lineage, no latency added	**Solid**	Real threaded worker, bounded queue, drop-oldest overflow policy
Golden set (5 cases)	Diverse edge cases	**Solid**	Includes blocking-severity prompt-injection & divide-by-zero cases
Golden set diff tool	Categorizes added/removed/modified	**Solid**	Correct, handles first-PR case, fully covered by tests
Wilcoxon comparator / gate	Detects aggregate degradation	**Solid**	Correct logic, handles zero-diff edge case, well tested
Drift detectors (PSI/KS/Z)	Statistically validated	**Solid**	Correct math, careful numerical handling (jitter, clipping, NaN)
Kappa calibration math	Rigorous statistical validation	**Solid**	Correct Cohen's Kappa, proper ordinal weighting
Unit test suite	Passing, typed, linted	**Mostly true**	26/26 pass, 88% coverage — but 0% on 3 key modules (below)
LLM provider call	Implied live in production	**Critical gap**	app/client.py: 15-line mock, no provider integration exists
LLM-as-judge scorer	Calibrated, trustworthy scorer	**Critical gap**	scorer.py: labeled STUB, pattern-matches prompt text, always near-perfect
Regression runner	Runs candidate prompts for real	**Critical gap**	runner.py: labeled STUB, returns an f-string, never calls feature code
CI regression gate	Blocks PRs on real degradation	**Critical gap**	on_prompt_change.yml passes the same file as baseline AND candidate
Failure analysis dashboards	Interactive triage for engineers	**Gap**	queries.py: 100% synthetic random data, zero DB queries
Drift scheduler	Cron job comparing to pinned baseline	**Gap**	Fetches np.random data, not DB; alerts via print(), not drift_events
DB indexes / constraints	Implied production-ready	**Gap**	Zero indexes, zero unique constraints anywhere in schema.sql

Area	Claimed (Summary)	Verified Status	Evidence
Secrets handling	Not addressed in summary	**Gap**	Hardcoded DB password in docker-compose.yml and in code fallback
requirements.txt	Fully typed, linted	**Gap**	scipy & numpy imported directly but absent from the file

1.2 Critical Finding: The Three-Part Stub Chain

This is the single most important finding in the audit and the reason Phase 9 is scoped the way it is. Three components, each explicitly labeled STUB in the source, chain together such that the CI regression gate — the core deliverable of Phase 5 — cannot currently fail, independent of whether a real API key is ever added.

(a) The judge scorer ignores the candidate output

monitoring/judge/scorer.py, _call_llm_judge():

```
def _call_llm_judge(prompt: str, model: str) -> str:
    """
    STUB: In a real environment, this calls the LLM API (e.g., Anthropic or OpenAI).
    For now, it returns a mocked successful response.
    """
    if "Score 1 if every numeric claim" in prompt:
        return '{"score": 1, "rationale": "All numeric claims match..."}'
    ...
    return '{"score": 1, "rationale": "Mock generic pass."}'
```

The function branches on substrings of the rubric prompt text, not on the candidate_output argument that is concatenated into that prompt. A memo that is factually wrong, policy-violating, or empty scores identically to a correct one, because the candidate text is never inspected. This is not a low-quality judge — it is not a judge at all yet.

(b) The regression runner never calls the real feature

monitoring/regression/runner.py, run_feature_evaluation():

```
def run_feature_evaluation(prompt_path: str, input_payload: Dict) -> str:
    """
    STUB: ... In reality, this would import the feature code and call it.
    """
    return f"Mock generated output for input: {input_payload}"
```

This function is what the regression suite calls to generate both the baseline and the candidate output. It does not import app/feature/credit_memo.py. It does not read prompt_path at all — the parameter is accepted and discarded. Every case in the golden set is scored against the same templated string, regardless of which prompt version is under test.

(c) The CI workflow compares a file to itself

.github/workflows/on_prompt_change.yml:

```
results = run_regression_suite(
    golden_set_path='monitoring/golden_set/data/v0001_credit_memo.jsonl',
    baseline_prompt_path='app/feature/prompts/credit_memo_v1.yaml',
    candidate_prompt_path='app/feature/prompts/credit_memo_v1.yaml',   # <- same file
    ...
# comment in the workflow: "In a real environment, baseline would point to
#  the deployed config. Here we just mock it using the same file..."
```

Two further problems compound this. First, the workflow triggers on changes to app/feature/prompts/**, but no such directory exists anywhere in the repository — the credit-memo prompts are hardcoded strings inline in credit_memo.py ("Find W2s", "Draft memo from docs", "Verify policy"), so this trigger path will never fire on a real prompt edit. Second, even if it did fire, baseline and candidate point at the same file, so there is structurally nothing to diff.

> **NET EFFECT**
>
> Stack (a) + (b) + (c) together: the runner generates the same fake string regardless of prompt, the judge scores that fake string based on rubric text rather than content, and the workflow feeds the comparator two identical inputs by construction. Even a candidate prompt that would catastrophically break the real feature in production would sail through this gate today. This is the load-bearing gap for the whole project's credibility, and it is what Phase 9, Workstream A closes.

1.3 Other Verified Gaps

Dashboards run on synthetic data, not the evidence database

dashboard/streamlit_app/queries.py contains one function per dashboard widget, and every single one is named mock_* and generates fresh random data (np.random.uniform, np.random.normal) on each page load. There is no SQLAlchemy session, no SQL query, and no import of the models in monitoring/db/models.py anywhere in the dashboard package. The five Streamlit pages render correctly and look complete, but none of them currently read a single row from run_traces, regression_runs, drift_events, or judge_versions.

The drift scheduler also stubs its data source and its alerting

monitoring/drift/scheduler_job.py's fetch_rolling_window_data() is labeled STUB and returns np.random.normal() draws instead of querying run_traces for the actual rolling window. Downstream, when a threshold is breached, the response is a print() statement — nothing is written to the drift_events table the schema was built to hold, so a real drift event today leaves no audit trail and cannot page anyone. Separately, the script reads config/thresholds.yaml as a bare relative path rather than resolving it via the module's own file location (the way config/model_pricing.yaml is correctly loaded in monitoring/tracing/decorators.py), so it will raise FileNotFoundError unless invoked from exactly the repository root.

expected_criteria is defined but never checked

Every golden set case carries a must_mention / must_not field (GoldenSetCase.expected_criteria), and it is faithfully threaded through the Pydantic schema and the database model. It is never read in

monitoring/judge/scorer.py or anywhere else. Right now the platform has no deterministic check layer at all — every evaluation, calibrated or not, goes through the LLM judge alone.

Database is missing indexes and uniqueness constraints

monitoring/db/schema.sql defines twelve tables with correct foreign keys but zero CREATE INDEX statements and zero UNIQUE constraints anywhere. Two concrete consequences: drift queries that filter run_traces by feature_name and scan by created_at (exactly what the scheduler needs to do once it's real) will full-scan the table as trace volume grows; and nothing at the database level stops two PromptVersion rows from being created with an identical content_hash, which quietly undermines the version-deduplication logic in resolve_prompt_version().

Only one Alembic migration exists

monitoring/db/migrations/versions/ contains a single revision, and its upgrade() body is op.execute() against the raw contents of schema.sql rather than an Alembic-generated diff against models.py. There is no evidence alembic revision --autogenerate has ever been run against the current SQLAlchemy models, so there's no guarantee the two are in sync, and no migration history exists to build on for future schema changes.

Secrets are hardcoded, not configured

The Postgres password (dev_password) appears twice: once in docker-compose.yml as a plaintext environment variable, and again as the literal fallback default inside async_writer.py's get_session() (os.getenv("DATABASE_URL", "postgresql+psycopg2://dev_user:dev_password@...")). There is no .env.example file anywhere in the repository, and no documented pattern for how a real deployment should supply credentials.

Dependency list is incomplete

requirements.txt lists SQLAlchemy, psycopg2-binary, alembic, pydantic, PyYAML, pytest, scikit-learn, statsmodels, streamlit, and pandas — but not scipy or numpy, both of which are imported directly in monitoring/regression/comparator.py (scipy.stats.wilcoxon), monitoring/drift/detectors.py (numpy, scipy.stats.ks_2samp), and monitoring/drift/scheduler_job.py. In this audit they were only present because pandas / statsmodels / scikit-learn happened to pull them in transitively — a fresh install with a stricter resolver, or a future version of those packages that drops the transitive dependency, would break the build with no warning until runtime. Separately, a .flake8 config file exists, but flake8 itself is not listed in requirements.txt, so the "linted" claim currently can't be verified by CI.

Cost tracking silently returns $0.00 for Anthropic models

config/model_pricing.yaml prices exactly two models, both OpenAI (gpt-4o-2026-03-01, gpt-3.5-turbo). compute_cost() in the tracing decorator falls back to {input_cost_per_1k: 0, output_cost_per_1k: 0} on a pricing miss rather than raising, so once real Anthropic calls are wired in during Phase 9, every trace will silently record $0.00 cost until this file is updated — a dashboard would show accurate token counts next to a cost column that quietly lies.

Test coverage: precise numbers, not "has tests"

The 26-test suite genuinely passes (verified by running pytest directly) and covers 88% of 526 statements across the modules that contain real logic. That aggregate number hides an important split, confirmed by running coverage with each file targeted individually:

Module	Coverage	Note
monitoring/db/models.py	100%	Fully exercised
monitoring/golden_set/schema.py	100%	Fully exercised
monitoring/golden_set/versioning.py	100%	Fully exercised
monitoring/regression/comparator.py	90%	Gate logic well covered
monitoring/judge/calibration/agreement.py	94%	Kappa math well covered
monitoring/drift/detectors.py	84%	Core statistics well covered
monitoring/tracing/decorators.py	72%	DB-failure fallback path under-tested
monitoring/regression/runner.py	**0%**	Never imported by any test
monitoring/drift/scheduler_job.py	**0%**	Never imported by any test
dashboard/ (entire package)	**0%**	Never imported by any test

The pattern is consistent and, in hindsight, predictable: every module built on real logic is well tested; every module built to orchestrate the stubs has no tests at all, because there is not yet a real behavior to assert against. There are also zero integration or end-to-end tests anywhere in the suite — all 26 tests exercise a single function in isolation.

One small tracing gap worth flagging

In app/feature/credit_memo.py, generate_credit_memo() — the top-level chain function — is not itself decorated with @traced_call; only its two sub-steps (retrieve_documents, check_memo) are. The middle step of the chain, the actual memo-drafting call, currently bypasses tracing entirely. This is a two-line fix but worth catching before it becomes a real blind spot in production trace data.

1.4 What Is Already Genuinely Solid

It's worth being equally precise about the other side of this. The following were verified as complete, correct, and ready to build on as-is — they do not need to be re-planned, only connected to real data:

- The evidence schema (12 tables) — normalization, foreign keys, and the append-only design are all sound.
- The async tracing pipeline — a real bounded-queue background writer with a sensible overflow policy, not a placeholder.
- All three drift detectors (PSI, KS, two-proportion Z) — correct statistics with careful numerical edge-case handling; these will work correctly the moment they receive real arrays instead of scheduler_job.py's random draws.
- The Wilcoxon comparator and severity-tiered gate decision logic — this is exactly the kind of nuanced blocking/major/statistical-warning logic a bank's model-risk function would want, and it already exists correctly.

- Cohen's Kappa calibration math, including correct ordinal weighting per dimension.
- The golden set itself and its diff tool — five well-designed cases spanning happy path, edge case, adversarial (prompt injection), and known-failure (divide-by-zero) categories, with a diff script that correctly categorizes changes for reviewers.
- The second CI workflow (on_golden_set_change.yml) — structurally sound and not dependent on any of the mocked components.
- Documentation depth for Phase 8 — three runbooks, an SR 11-7 compliance mapping, and an architecture doc, roughly 2,000 words of real operational content.

2. Phase 9 — Real-Signal Integration

The requested "next-to-final" development phase.

Phase 9's job is narrow and specific: replace the three labeled stubs with real behavior, in an order where each piece can be verified before the next depends on it, and close the infrastructure gaps that would otherwise undermine the result. Nothing in this phase is about adding new capability — Phases 1–8 already designed the right capabilities. This phase is about making the existing design true.

> **BUDGET-AWARE BY DESIGN**
> Every workstream below is written for free-tier Anthropic API usage: small golden set, low request volume, aggressive caching of repeated calls, and a hard per-run token/request ceiling enforced in code (not just documented) so a CI misconfiguration can't burn through a monthly quota unattended.

2.1 Workstream Order

#	Workstream	Depends On	Unblocks
A	Real LLM provider client	—	B, C
B	Real judge (real calls + real human labels)	A	C, D
C	Real regression runner + fixed CI baseline/candidate	A, B	D, E
D	Deterministic expected_criteria checks	B	C (adds a second signal)
E	Dashboards wired to the real database	A (needs real trace rows to show)	Phase 10
F	Drift scheduler wired to the real database	A	Phase 10
G	Infrastructure hardening (indexes, secrets, deps, cost config)	—, can run in parallel	Phase 11

2.2 Workstream A — Real LLM Provider Client

Replaces

app/client.py's mock_llm_call (currently 15 lines, no network call, deterministic fake output).

Build

- A real `anthropic` SDK client wrapping `messages.create()`, returning the same (output_text, usage_dict) tuple contract the rest of the system already expects — this keeps the change isolated to one file.
- Provider selection via a MODEL_PROVIDER config value, defaulting to a small, inexpensive Claude model suitable for this workload — keep this configurable rather than hardcoded so the free-tier constraint doesn't get baked into the architecture.

- Retry with exponential backoff on transient errors (429, 5xx) — 3 attempts, capped, since free-tier keys are the most likely to hit rate limits during a CI burst.
- A response-content cache keyed on a hash of (prompt, model_config), scoped to the regression-run process — the golden set is only 5 cases, but each regression run scores baseline and candidate across multiple rubric dimensions, and repeated dev-loop runs against unchanged prompts shouldn't re-spend quota.
- A hard per-run request ceiling (e.g. `MAX_LLM_CALLS_PER_RUN`), enforced in code with a clear exception when exceeded — this is the free-tier safety net, not just a comment.
- Real error propagation instead of swallowing — the caller (the @traced_call decorator) already logs and records error in the trace row, so the client should raise, not return a fake success.

Also fix while touching this file

- `credit_memo.py`: decorate `generate_credit_memo()` itself with `@traced_call`, and route its "outer" draft-memo call through the same client function as the other two steps instead of calling the client inline — closes the untraced-middle-step gap found in the audit.
- `config/model_pricing.yaml`: add real entries for whichever Claude model(s) get used, so `compute_cost()` stops silently returning $0.00 the moment real calls start flowing.

Done when

> **VERIFICATION**
> A manual call to generate_credit_memo() with a real input produces a genuinely different memo for two meaningfully different inputs (verify by eye, not just by non-null output).
> A run_traces row appears in Postgres with a non-zero cost_usd, real token counts, and real latency_ms.
> Deliberately exhausting MAX_LLM_CALLS_PER_RUN raises a clear, catchable exception rather than silently truncating.

2.3 Workstream B — A Judge That Actually Reads the Output

Replaces

monitoring/judge/scorer.py's _call_llm_judge (pattern-matches on rubric prompt text, ignores candidate_output) and monitoring/judge/calibration/mock_human_labels.json (synthetic ground truth).

Build — the scorer

- Route _call_llm_judge through the same provider client built in Workstream A, using structured output (a strict JSON schema for {score, rationale}) so parsing failures become rare rather than something score_output has to defensively catch on every call.
- Keep the existing per-dimension, separate-call design in score_output — this is a deliberate and correct choice already in the code (it mitigates position and verbosity bias by not asking one giant prompt to judge five things at once) and should not be simplified away.
- Add a distinct, lower-temperature or fixed-seed configuration for the judge model versus the feature model, since judge consistency matters more than judge creativity.

Build — real human calibration labels

This is the part most likely to be skipped under time pressure, and it's the part that makes the word "calibrated" mean something. Cohen's Kappa measures agreement between the judge and a genuine second rater — computing it against synthetic labels validates nothing except that the mock data was internally consistent.

- Take the 5 existing golden set cases (and expand modestly — see Workstream D / Phase 10 for golden set growth) and have a real person score each dimension by hand, independently, before ever looking at the judge's scores — this ordering matters for avoiding anchoring bias.
- Store these as the new, real monitoring/judge/calibration/human_labels.json, replacing the mock file outright rather than adding alongside it, so there's no ambiguity about which file agreement.py should read.
- Run `monitoring/judge/calibration/agreement.py` — this file needs no changes; the Kappa math was already verified correct during the audit — and record the resulting `JudgeVersion.calibration_status` and `last_kappa_score` for real in the database.

Build — deterministic checks (Workstream D, folded in here)

GoldenSetCase.expected_criteria (must_mention / must_not) is already defined and stored but never checked. Add a small, separate function — deterministic, not LLM-based — that checks these lists against candidate_output with simple substring or regex matching, and surface its result as an additional signal alongside the LLM judge's scores rather than folding it into the same dimension. A cheap deterministic check that a memo mentions "DSCR" catches a category of failure (the model dropping a required term or section) that an LLM judge can occasionally miss or rationalize past, and it costs zero API calls.

Done when

> **VERIFICATION**
> Feed the judge two candidate outputs for the same case — one correct, one deliberately wrong (e.g. an invented DSCR figure) — and confirm the scores differ. This single test is the most important one in the whole phase: it is the thing that was structurally impossible before this workstream.
> human_labels.json contains scores from an actual person, not generated data, and calibration_report() produces a real kappa per dimension against it.
> At least one dimension's real kappa is below its configured threshold at first pass — a first calibration run that passes every dimension immediately is itself a signal to double check the rubric or the human labels, not a milestone to celebrate uncritically.

2.4 Workstream C — A Regression Runner That Regresses

Replaces

monitoring/regression/runner.py's run_feature_evaluation (returns an f-string, ignores prompt_path) and the same-file baseline/candidate bug in .github/workflows/on_prompt_change.yml.

Build — prompts as real, diffable artifacts

The CI workflow already watches app/feature/prompts/**, which is the right instinct — it just points at a directory that doesn't exist, because the real prompts are hardcoded strings inside credit_memo.py. Fix the cause, not the trigger path: extract the three inline prompt strings into versioned YAML files (e.g. app/feature/prompts/credit_memo_v1.yaml), and have credit_memo.py load from there. This is what

makes "baseline vs. candidate" a meaningful, file-diffable concept instead of two Python identifiers pointing at the same code.

Build — run_feature_evaluation for real

- Import and call the actual generate_credit_memo (or its sub-steps, depending on what a given golden set case is meant to exercise) rather than fabricating a string — this function's entire job is to be a thin, testable seam between the golden set loader and the real feature code.
- Load the prompt template from prompt_path (now a real file per the fix above) and pass it through to the feature call — this is the one line that makes baseline and candidate genuinely different when the workflow is fixed.
- Preserve the existing N-repeat / median-score structure in run_regression_suite() exactly as designed — the code already anticipates stochasticity absorption via repeated runs; Phase 9 just needs repeats set to a real value (2–3) instead of the placeholder default of 1, budgeted against the free-tier call ceiling from Workstream A.

Fix — the CI workflow itself

```
# on_prompt_change.yml, corrected shape:
results = run_regression_suite(
    golden_set_path='monitoring/golden_set/data/v0001_credit_memo.jsonl',
    baseline_prompt_path=get_deployed_prompt_path('credit_memo'),  # main branch version
    candidate_prompt_path=get_changed_prompt_path(pr_diff),        # PR branch version
    rubric_path='monitoring/judge/rubric/credit_memo_rubric_v2.yaml',
    repeats=2,
)
```

The exact mechanism for "deployed prompt" can be as simple as checking out the base-branch version of the same YAML file the golden-set-diff workflow already does (git checkout ${{ github.event.pull_request.base.sha }}) — the second workflow in this repo already demonstrates the correct pattern for fetching an old-versus-new file across a PR boundary; Workstream C's fix largely means applying that existing pattern to the regression workflow instead of inventing a new one.

Done when

> **VERIFICATION**
>
> Open a test PR that changes one word in a prompt YAML file in a way that should degrade output quality (e.g. remove the instruction to compute DSCR). Confirm the workflow trigger fires — it does not today, since the watched path doesn't exist.
> Confirm the regression run reports a real, non-identical baseline_score and candidate_score for at least one golden set case.
> Confirm the gate_decision is "block" for that PR, and that the PR comment (already wired via report_builder.py) shows the specific failing case. This is the moment the project's core claim — "a silent quality drop after a prompt change is caught before merge" — becomes true for the first time.

2.5 Workstream E — Dashboards on Real Data

Replaces

Every mock_* function in dashboard/streamlit_app/queries.py.

- Replace each mock_* function with a SQLAlchemy query against the corresponding table — mock_run_traces() → a query.filter(RunTrace.feature_name == feature) against run_traces, mock_regression_history() → a query against regression_runs joined with regression_case_results, mock_drift_events() → a direct read of drift_events, mock_calibration_status() → a read of the latest JudgeVersion row.
- Add a dashboard/streamlit_app/__init__.py — this is also what's needed to make mypy able to check the dashboard package at all; it currently errors out immediately on a module-naming collision before checking a single line.
- Keep function names and return shapes (DataFrames with the same columns) identical where practical — this keeps the diff small and reviewable, and confirms the five page files themselves don't need to change, only their data source.
- Add a visible "no data yet" empty state for each page rather than letting an empty query result render a blank or broken chart — with a 5-case golden set and a fresh Phase 9 rollout, empty states will be the first thing anyone actually sees.

2.6 Workstream F — Drift Scheduler on Real Data

Replaces

monitoring/drift/scheduler_job.py's fetch_rolling_window_data (random draws) and its print()-only alerting.

- Replace `fetch_rolling_window_data()` with a real query against `run_traces` filtered to the requested rolling window, feature, and metric — this becomes straightforward once Workstream G's index on `(feature_name, created_at)` is in place.
- When a detector crosses its threshold, write a real row to `drift_events` (the schema and model already fully support this) instead of printing — this is what makes a drift event a persistent, auditable artifact rather than a line in a CI log that scrolls away.
- Fix the relative-path bug: load `config/thresholds.yaml` the same way `load_pricing()` already correctly does — resolved via `os.path.dirname(__file__)` — so the scheduler doesn't silently fail depending on invocation directory.
- Leave detectors.py itself untouched — it was verified correct and is already fully decoupled from the data-source stub; this workstream is purely about what feeds it.

2.7 Workstream G — Infrastructure Hardening

Smaller, independent fixes surfaced by the audit. Each is quick individually; grouped here so none get lost.

File	Fix
monitoring/db/schema.sql	Add indexes on run_traces(feature_name, created_at), judge_scores(run_trace_id), drift_events(feature_name, window_start). Add a UNIQUE constraint on prompt_versions(content_hash).
New Alembic migration	Generate a real second revision via alembic revision --autogenerate once the index/constraint changes above land in models.py, establishing an actual migration history instead of one raw schema dump.

File	Fix
docker-compose.yml	Move the Postgres password to an environment-variable reference; add a .env.example documenting every variable the compose file and the app expect.
async_writer.py	Remove the hardcoded dev_password fallback from get_session()'s DATABASE_URL default — fail loudly if the env var is unset in anything other than an explicit local-dev mode.
requirements.txt	Add scipy and numpy explicitly (both already in active use, currently present only by transitive luck). Add flake8, since a .flake8 config already exists with no way to run it.
docker-compose.yml	Either wire the langfuse-server service to the real tracing pipeline or remove it — right now it starts and does nothing, which reads as unfinished to anyone who runs docker compose up and wonders what it's for.
config/model_pricing.yaml	Add real pricing entries for the Claude model(s) selected in Workstream A (covered above, listed again here for the consolidated checklist).

3. Phase 10 — Verification & Testing

The existing 26 unit tests are real and worth keeping exactly as they are — they correctly pin down the statistics and schema logic. What Phase 10 adds is everything the audit found missing: integration tests that exercise the seams between modules (which is precisely where the stub chain was hiding), load behavior under something closer to real traffic, and a security pass appropriate for a system that will eventually touch financial-document content.

3.1 Integration & End-to-End Tests

Zero integration tests exist today. Every one of the 26 current tests calls a single function with hand-constructed inputs. That's exactly the kind of test suite that can pass 26/26 while the CI gate underneath it is structurally unable to fail — the audit confirmed this is precisely what happened. The tests below are the ones designed to catch that class of problem specifically.

- `tests/integration/test_full_trace_lifecycle.py` — call `generate_credit_memo()` for real (against a test/sandbox model config), then assert a matching row actually lands in `run_traces` in a test Postgres instance, with a non-null cost_usd and correct `parent_span_id` linkage across the multi-step chain.
- `tests/integration/test_judge_detects_bad_output.py` — the single highest-value new test in the whole suite. Feed the real judge one correct and one deliberately corrupted candidate memo for the same golden set case and assert the scores differ meaningfully. This test would have failed against the pre-Phase-9 code and is the direct regression guard against ever silently reintroducing a scorer that ignores its input.
- `tests/integration/test_regression_gate_blocks_real_regression.py` — construct two genuinely different prompt YAML files, one better and one deliberately worse, run `run_regression_suite()` against both, and assert the gate decision is `block` for the worse one and `allow` for an unchanged prompt. This is the automated version of the manual CI verification from Workstream C, made permanent.
- `tests/integration/test_drift_scheduler_writes_events.py` — seed a reference distribution and a current window with a deliberate shift, run `evaluate_drift()`, and assert a real row appears in `drift_events` — catching any regression back toward print()-only alerting.
- `tests/integration/test_dashboard_queries_against_seeded_db.py` — seed a handful of realistic rows into a test database and assert each `queries.py` function returns data shaped correctly from those rows, not from `np.random`.
- A minimal end-to-end smoke test that runs the full chain — generate a memo, score it, run a regression comparison, evaluate drift, and confirm the dashboard can render something from the result — as one script, intended to be the fastest possible "is the whole system alive" check before a deploy.

3.2 Golden Set Expansion

Five cases is enough to prove the pipeline works, but it's thin for a system whose entire pitch is thorough coverage of edge cases and adversarial inputs. Target 25–40 cases before calling this production-grade, keeping the existing category and severity structure:

- **Happy path** (5–8 cases): straightforward inputs across different industries and deal sizes, to catch basic prompt regressions.
- **Edge case** (8–12 cases): the kind already represented — seasonal income, multi-entity structures — expanded to cover missing financials, negative NOI, extreme DSCR ratios, and currency/locale variations.
- **Adversarial** (5–8 cases): expand beyond the existing prompt-injection case to include attempts to make the model state a compliance-violating recommendation, leak the rubric itself, or override its instructions via injected text inside financial document content.
- **Known failure** (5–8 cases): expand beyond divide-by-zero to include any failure mode discovered during Phase 9/10 testing — every bug found during integration testing is a candidate golden set case, since a bug that broke it once can break it again after a future prompt change.

Each new case needs the same human-labeled calibration pass described in Workstream B before it counts toward Kappa — expanding the golden set without expanding the human labels alongside it would leave newly added cases unvalidated against the very judge that's supposed to score them.

3.3 Load & Resilience Testing

The tracing pipeline (async_writer.py) is a single in-process daemon thread with a bounded queue and a drop-oldest overflow policy. That's a reasonable design for a single-process demo; it has real, known limits worth deliberately testing rather than discovering in production:

- Queue saturation test: fire enough concurrent traced calls to exceed the 1,000-item queue bound and confirm the drop-oldest behavior triggers cleanly (a warning is logged, no exception propagates to the caller) rather than blocking or crashing the request path.
- Multi-process behavior: the current design has no cross-process coordination — if the feature is ever run behind multiple worker processes (e.g. gunicorn with more than one worker), each process gets its own queue and its own daemon thread. Document this limitation explicitly and decide, based on real deployment shape, whether Phase 11 needs a shared external queue (e.g. Redis-backed) or whether single-process is an acceptable constraint for the scale this project targets.
- DB-down resilience: test_kill_the_db_resilience already exists and passes — extend it into an integration-level test that also verifies the trace queue keeps accepting new items (rather than backing up unboundedly) while Postgres is unreachable, and drains correctly once it recovers.
- Regression suite timing under real API latency: with Workstream A's real provider calls in place, measure actual wall-clock time for a full golden-set regression run at the planned repeat count, and confirm it fits comfortably inside GitHub Actions' default job timeout — this was a non-issue when run_feature_evaluation() was instant, and won't be once it makes real network calls.

3.4 Security Review

This system is designed to sit next to a bank's credit-memo drafting workflow. The security bar should match that, even for a portfolio project — and it doubles as evidence of exactly the kind of judgment the SR 11-7 framing is meant to demonstrate.

- Prompt injection resilience: the golden set already includes one adversarial prompt-injection case — once the judge and runner are real (Phase 9), formally verify the feature's behavior against it and expand this

category as described in 3.2, since this is the single most relevant attack surface for an LLM feature that ingests uploaded financial documents.

- Secrets audit: confirm no API keys, database credentials, or the NEXTAUTH_SECRET / SALT values currently hardcoded in docker-compose.yml ever reach version control after the Workstream G fixes — add a pre-commit hook or CI step (e.g. gitleaks or truffleHog) scanning for accidentally committed secrets, not just a manual check.
- Dependency vulnerability scan: run pip-audit or an equivalent against the corrected requirements.txt from Workstream G, since it hasn't been checked at all yet — this is a five-minute step with real signal for a finance-adjacent project.
- SQL injection surface: the SQLAlchemy ORM usage throughout the codebase is parameterized by default (verified during the audit — no raw string-interpolated SQL was found anywhere), so this is largely a confirmation pass, not new work — but the dashboard queries added in Workstream E are new code and should be reviewed with this specifically in mind before they ship.
- PII / sensitive-data handling: golden set cases and real traces will contain synthetic (or, if using any real-shaped test data, sanitized) financial figures — explicitly document in the compliance mapping that no real customer PII is used anywhere in this portfolio version, since that's a real distinction a reviewer or interviewer would reasonably ask about.

4. Phase 11 — Production Deployment

This phase takes the verified-real system from a local docker-compose setup to something reachable on the internet, with real (if modest) hosting. The goal is a genuine, live, demoable deployment — not a Kubernetes cluster sized for a bank's actual traffic, which would be disproportionate for a portfolio project and isn't what makes this credible to a reviewer. What makes it credible is that it's real, it's running, and every claim in the README (Phase 12) can be clicked through and verified live.

4.1 Hosting Shape

Recommended shape, chosen for being cheap-to-free, fast to stand up, and legible to anyone reviewing the project (an interviewer can find and understand a Render/Railway/Fly.io deployment far faster than a bespoke Terraform module):

Component	Suggested Home	Why
PostgreSQL (evidence DB)	Managed free/hobby tier — e.g. Supabase, Neon, or Railway Postgres	Automatic backups and connection pooling without managing a DB server by hand
Streamlit dashboards	Streamlit Community Cloud, or a small Render/Railway web service	Streamlit Cloud is free and purpose-built for exactly this
CI (regression & drift gates)	GitHub Actions (already in place)	No change needed — already the right tool
Drift scheduler	GitHub Actions on a cron schedule, or Railway's cron jobs	Avoids paying for an always-on worker for a low-frequency job
Secrets	GitHub Actions Secrets + host platform's env var manager	No custom secrets infrastructure needed at this scale

4.2 Environment & Configuration

- Formalize three environments — `local`, `ci`, `production` — each with its own DATABASE_URL and its own model-pricing / rate-limit configuration, so a CI run can never accidentally point at the production database.
- Ship the .env.example from Phase 9's Workstream G as the actual onboarding document for this step — anyone (including a future you) standing this up from scratch should be able to go from a fresh clone to a running local instance using only that file and the README.
- Run the (by now real, multi-revision) Alembic migration chain against the production database as an explicit, logged step — `alembic upgrade head` — never by hand-running `schema.sql` again, which is exactly the shortcut the current single-migration setup was taking.
- Set real, conservative values for MAX_LLM_CALLS_PER_RUN and the drift scheduler's polling frequency in the production environment specifically, distinct from whatever's convenient for local dev — this is where the free-tier budget constraint actually gets enforced end to end.

4.3 CI/CD to Production

- Add a deploy workflow, separate from the existing regression-gate and golden-set-diff workflows, triggered on merge to main — the existing two workflows are validation gates and should stay exactly as they are; deployment is a third, distinct concern.
- Sequence the deploy workflow as: run the full test suite (unit + Phase 10 integration tests) → run Alembic migrations against production → deploy the dashboard service → run the end-to-end smoke test from Phase 10 against the live production URL → only then consider the deploy complete.
- Make the regression gate from Workstream C a required check on the branch protection rule for main, not just an informational PR comment — this is a small settings change that's also the difference between "we built a gate" and "the gate actually gates."

4.4 Observability in a Live Environment

The failure-analysis dashboards (Phase 7) are the primary observability surface here, but a live deployment benefits from a couple of things below the dashboard layer too:

- Structured logging (JSON logs) from the tracing writer and drift scheduler, so host-platform log search is actually useful when something needs debugging live, rather than relying on grepping through print statements.
- A minimal uptime/health check endpoint for the dashboard service, and a scheduled ping (e.g. a free UptimeRobot monitor) — mostly so a demo link doesn't quietly go stale between when it's built and when someone clicks it from a resume.
- Wire the drift scheduler's critical-severity alerts (Workstream F) to a real, low-friction notification channel appropriate for a portfolio project's scale — a webhook to a Slack channel or even email is enough to demonstrate the concept without building paging infrastructure sized for an actual bank's on-call rotation.

5. Phase 12 — Resume & Portfolio Packaging

The engineering in Phases 9–11 is what makes this project true. Phase 12 is what makes it legible to someone who has five minutes to evaluate it — a recruiter skimming a resume link, a hiring manager clicking through before a call, an interviewer asking "walk me through this." Right now the repository's README is a single line (just the project title), which undersells everything else that's actually been built.

5.1 README Rewrite

Replace the current one-line README with a structure that front-loads what a skimming reader needs, in this order:

1. A one-paragraph summary of what the system does and why it exists (the SR 11-7 / silent-quality-drop framing already in the project summary is strong — lead with it).
2. A live demo link (the deployed dashboard from Phase 11) and, ideally, a 60–90 second screen recording embedded or linked, showing a PR triggering the regression gate and the dashboard reflecting a real trace.
3. An architecture diagram (a simple one is fine — the existing docs/architecture.md has the content; it just needs a visual alongside the prose).
4. A "what's actually implemented" section that is specific and honest — this project already has the raw material for genuine standout claims (real statistical tests, a real calibrated judge, a real CI-blocking gate) and doesn't need embellishment; specificity is more convincing than superlatives to the technical readers who matter most here.
5. Setup instructions that route through the .env.example from Phase 9, tested by actually following them on a clean checkout before publishing.
6. A short "design decisions" section — 3–5 choices worth explaining, e.g. why per-dimension judge calls instead of one combined call, why Wilcoxon instead of a simple mean-difference threshold, why PSI for score drift specifically. These are exactly the questions a good interviewer asks, and having them answered in writing is preparation, not just documentation.

5.2 Demo Video / Walkthrough

A short recorded walkthrough is worth disproportionately more than its length suggests, because it's the artifact most likely to actually get watched. Suggested script:

- 0:00–0:15 — one sentence on the problem (banks need audit-grade evidence that an LLM feature didn't silently degrade).
- 0:15–0:45 — open a PR that changes a prompt for the worse; show the regression gate actually blocking it, with the specific failing case visible in the PR comment.
- 0:45–1:15 — flip to the live dashboard, show a real trace with real cost/latency numbers, and the drift monitor view.
- 1:15–1:30 — one sentence on what's next / what you'd build with more time, which signals maturity and self-awareness rather than presenting the project as flawlessly finished.

5.3 Resume Bullets

Grounded specifically in what the audit verified as real, so every number here is defensible if an interviewer asks a follow-up question:

- Built an LLM evaluation and observability platform for a simulated bank credit-memo feature, implementing a calibrated LLM-as-judge (Cohen's Kappa against human-labeled ground truth) and a CI-blocking regression gate using paired Wilcoxon significance testing across a versioned golden test set.
- Designed a 12-table append-only PostgreSQL evidence schema with full lineage from prompt version through judge score to drift event, mapped explicitly to Federal Reserve SR 11-7 model-risk governance requirements.
- Implemented production drift detection using three complementary statistical tests (Population Stability Index, Kolmogorov-Smirnov, two-proportion Z-test) to separately monitor judge-score, output-length, and refusal-rate distributions against a pinned baseline.
- Built an async, non-blocking tracing pipeline capturing full request lineage, token usage, and cost per LLM call with zero added latency to the calling application, achieving 88%+ test coverage on core logic.

(Adjust exact figures — Kappa scores, case counts, coverage percentage — to whatever Phase 9/10 actually produce; these bullets are a template, not a promise to hit specific numbers.)

5.4 How to Talk About This Honestly

> **THE HONEST VERSION IS THE STRONGER VERSION**
>
> This project's real strength — a genuinely correct statistical evaluation and drift-detection architecture — is more impressive than a system that quietly worked first try, because a system that worked first try wouldn't need someone who understands why Wilcoxon over a mean-difference threshold, or why PSI bins need quantile jitter to avoid degenerate bins. If asked "what was hardest" or "what would you do differently," the true answer is exactly the finding at the center of this whole roadmap: getting the mocked judge and the mocked regression runner replaced with real signal, and discovering along the way that the CI gate had been comparing a file to itself. That is a genuinely good story, told straight, and it's a far better answer than pretending the project shipped complete on the first pass.

Concretely: it's fine, and arguably better, to describe this project's history as "I designed and built the full evaluation architecture first, including the statistics and the schema, then wired in real model calls and fixed the CI gate as a distinct second phase" — that is what actually happened, it's a defensible and common way real projects get built, and it demonstrates the ability to design a system correctly before all its parts exist, which is its own real skill.

Appendix — Consolidated File-Change Checklist

Every file touched across Phases 9–11, in one place, for use as an implementation tracker. Ordered by workstream.

Phase 9

File	Change
app/client.py	Replace mock_llm_call with real Anthropic client, retry/backoff, cache, call ceiling
app/feature/credit_memo.py	Decorate generate_credit_memo with @traced_call; route outer call through client
app/feature/prompts/*.yaml (new)	Extract hardcoded prompt strings into real, versioned, diffable files
monitoring/judge/scorer.py	Real _call_llm_judge via provider client with structured JSON output
monitoring/judge/calibration/human_labels.json (new, replaces mock)	Real human-scored labels for Kappa calibration
monitoring/judge/checks.py (new)	Deterministic expected_criteria (must_mention/must_not) checker
monitoring/regression/runner.py	Real run_feature_evaluation calling the actual feature code
.github/workflows/on_prompt_change.yml	Fix baseline vs. candidate to diff real base-branch vs. PR-branch prompt files
dashboard/streamlit_app/queries.py	Replace every mock_* with a real SQLAlchemy query
dashboard/streamlit_app/__init__.py (new)	Fixes mypy module-collision error on the dashboard package
monitoring/drift/scheduler_job.py	Real fetch_rolling_window_data query; write real drift_events rows; fix config path
monitoring/db/schema.sql + models.py	Add indexes and unique constraints
monitoring/db/migrations/versions/ (new revision)	Real Alembic autogenerate revision, not a raw schema dump
docker-compose.yml	Env-var secrets, not hardcoded password; resolve or remove orphaned langfuse-server
async_writer.py	Remove hardcoded password fallback; fail loudly if DATABASE_URL unset
requirements.txt	Add scipy, numpy, flake8 explicitly
config/model_pricing.yaml	Add real Claude model pricing entries
.env.example (new)	Document every required environment variable

Phase 10

File	Change
tests/integration/ (new directory, 5+ files)	Full-lifecycle trace, judge-detects-bad-output, gate-blocks-real-regression, drift-writes-events, dashboard-reads-seeded-db
monitoring/golden_set/data/v0002_credit_memo.jsonl (new)	Expand golden set from 5 to 25–40 cases across all four categories
tests/load/ (new)	Queue-saturation and DB-down resilience tests under real concurrency
Security pass	Secrets scan (gitleaks), dependency scan (pip-audit), prompt-injection case review

Phase 11

File	Change
.github/workflows/deploy.yml (new)	Test → migrate → deploy → smoke-test pipeline, separate from the validation gates
Branch protection settings	Make the regression gate a required check on main
Hosting config (host-specific)	Managed Postgres, Streamlit Cloud or equivalent, cron for drift scheduler

Closing Note

The eight phases already built here represent real, substantial engineering — a correctly normalized evidence schema, genuinely correct statistics for regression gating and drift detection, a careful async tracing pipeline, and thorough compliance documentation. None of that needs to be redone. What separates the current state from a system that can honestly back every claim in its own project summary is the real-signal integration scoped as Phase 9, verified by the testing scoped as Phase 10, and made live by the deployment scoped as Phase 11. Phase 12 is what turns all of that into something a resume and an interview can stand on without any claim needing a footnote.